Министерство науки и высшего образования Российской Федерации
Федеральное государственное бюджетное образовательное учреждение
высшего образования
«Московский государственный технический университет имени Н.Э. Баумана
(национальный исследовательский университет)»
(МГТУ им. Н.Э. Баумана)

ФАКУЛЬТЕТ «Головной учебно-исследовательский и методический центр профессиональной реабилитации лиц с ограниченными возможностями здоровья (инвалидов)»

КАФЕДРА «Компьютерные системы автоматизации производства (РК9)»

РАСЧЕТНО-ПОЯСНИТЕЛЬНАЯ ЗАПИСКА

К ВЫПУСКНОЙ КВАЛИФИКАЦИОННОЙ РАБОТЕ

НА ТЕМУ:

«Программное средство удаленного управления роботом-манипулятором в прямоугольной системе координат»

Студент РК9Ц-101Б _____ Д.А. Дубровский

Руководитель ВКР _____ С.А. Буханов
(Подпись, дата)

Нормоконтролер _____ …
(Подпись, дата)

2022 г.

РЕФЕРАТ

Пояснительная записка содержит 54 страницы, 3 таблицы, 21 рисунок, 2 приложения и 17 источников информации.

Ключевые слова: RV-2AJ, промышленный робот, C#, UML-диаграммы, функциональная и структурная схемы, алгоритм удаленного управления роботом.

Выпускная квалификационная работа состоит из Введения, шести глав, Заключения, Списка использованных источников. Во введении обоснована актуальность темы выпускной квалификационной работы, сформулированы цель работы и задачи исследования. В первой главе проводиться анализ существующих систем автоматизации, анализируется промышленный робот-манипулятор, сравниваются системы координат. Во второй главе разрабатываются необходимые схемы. Третий раздел является техническим заданием. Четвертая глава предусматривает определение метода и протокола подключения робота, разработка функциональной и структурной схемы. В пятом разделе производится рабочее проектирование. 6 раздел – апробация результатов. Результаты представлены с помощью рисунков и схем. Подчеркнуты основные особенности функционирования схемы.

В работе применялись следующее программное обеспечение: текстовый редактор Microsoft Word 2016, сервисы построения графиков и блок-схем draw.io, BPwin 4.0, среда разработки Microsoft Visual Studio 2022.

СОДЕРЖАНИЕ

РЕФЕРАТ 2
ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ 5
ВВЕДЕНИЕ 6
1. ПРЕДПРОЕКТНОЕ ОБСЛЕДОВАНИЕ РОБОТА-МАНИПУЛЯТОРА . 8
1.1 Анализ состояния проблемы по информационным источникам 8
1.2 Целевое обследование робота-манипулятора 9
1.2.1 Анализ устройства промышленного робота 9
1.2.2 Анализ системы координат промышленного робота 11
1.3 Анализ состояния объекта 13
1.3.1 Анализ функциональных возможностей 13
1.3.2 Определение команд, необходимых для управления роботом в прямоугольной системе координат 14
2. КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ СИСТЕМЫ 26
2.1 Выбор общесистемной методологии проектирования 26
2.2 Выбор CASE-средства проектирования 27
2.3 Разработка UML диаграмм 28
2.3.1 Разработка Use-case диаграммы 28
2.3.2 Разработка диаграммы компонентов 30
2.4 Разработка функциональной структуры системы 30
3. ТЕХНИЧЕСКОЕ ЗАДАНИЕ 33
4. СТРУКТУРНОЕ (ТЕХНИЧЕСКОЕ ПРОЕКТИРОВАНИЕ) УПРАВЛЕНИЯ РОБОТОМ 40
4.1 Разработка функциональной и структурной схемы 40
4.2 Разработка блок-схемы основных алгоритмов работы программы 41
5. РАБОЧЕЕ ПРОЕКТИРОВАНИЕ 44

5.1 Разработка интерфейса программы 44
5.1 Разработка программного кода..... 46
6. АПРОБИРОВАНИЕ РЕЗУЛЬТАТОВ..... 49
ЗАКЛЮЧЕНИЕ 51
СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ 52
ПРИЛОЖЕНИЕ А 55
ПРИЛОЖЕНИЕ Б..... **Ошибка! Закладка не определена.**~~69~~

ОБОЗНАЧЕНИЯ И СОКРАЩЕНИЯ

АСУ ТП – автоматизированная система управления технологическим процессом

АСОИУ – автоматизированная система обработки информации и управления

ИС – информационная система

UML (Unified Modeling Language) – унифицированный язык моделирования

CASE (Computer-Aided Software Engineering) – система автоматизированной разработки программ

IDEF (I-CAM DEFinition) – методология функционального моделирования

RS-232 (Recommended Standard 232) – стандарт физического уровня для асинхронного интерфейса

ПК – персональный компьютер

ТСА – технические средства автоматизации

ОС – операционная система

ПО – программное обеспечение

ТЗ – техническое задание

ВВЕДЕНИЕ

В нынешнее время большинство сфер экономики, так или иначе, нуждаются в автоматизировании производственных процессов, замене ручного труда на более эффективные и экономически выгодные решения.

Актуальность работы связана с тем, в условиях импортозамещения, возникает потребность как в разработке отечественных автоматизированных комплексов, способных увеличивать эффективность, объемы производства и качества труда, так и в специалистах, способных разрабатывать программные системы, повышая качество выпускаемых товаров, разрабатывая новые алгоритмы. Данный программный комплекс вносит некоторую научную новизну в вопросе разработки и автоматизации производственных процессов.

Целью данной работы является создание универсального программного средства для управления роботом-манипулятором в прямоугольной системе координат, для достижения компетенций, сформированных в процессе обучения.

В ходе выполнения квалификационной работы были поставлены следующие ***задачи***:

- Проанализировать устройство и систему координат промышленного робота.
- Проанализировать систему команд языка управления роботом.
- Определить команды, необходимые для управления роботом в прямоугольной системе координат.
- Определить метод и протокол подключения персонального компьютера к контроллеру робота.
- Разработать алгоритм системы удаленного управления роботом-манипулятором в прямоугольной системе координат.
- Разработать исходный код.
- Разработать интерфейс пользователя системы.

- Протестировать работу системы.

Типовым предметом данной работы является проектирование и разработка системы для управления роботом-манипулятором в прямоугольной системе координат.

В соответствии с поставленной целью и задачами исследования в работе были определены следующие ***методы***: анализ состояния проблемы по информационным источникам, анализ устройства, анализ системы координат анализ состояния объекта, проведено сравнение функциональных возможностей, проведена конкретизация разработки.

Объектом автоматизации выступает 5-осевой робот-манипулятор RV-2AJ от фирмы Mitsubishi. Так, линейка роботов оснащена всеми основными эксплуатационными характеристиками робота такими как, мощь скорость и компактность. Данные роботы уверенно отвечают потребностям нынешней эпохи и значительно продвинули вперед АСУ ТП [1].

Внедрение систем автоматизированного управления промышленными роботами, позволяет автоматизировать многие производственные процессы и значительно упростить контроль и управление за счет разработки и модификации управляющих программ для промышленных роботов.

1. ПРЕДПРОЕКТНОЕ ОБСЛЕДОВАНИЕ РОБОТА-МАНИПУЛЯТОРА

1.1 Анализ состояния проблемы по информационным источникам

В настоящее время существует острая потребность обеспечения интеллектуальных систем программными продуктами в условиях импортозамещения. Так, обрабатывающая промышленность должна справляться с меняющимися проблемами, которые связаны с глобальной конкуренцией, занятостью, закупками и другими. Такие задачи можно решить при помощи автоматизированных роботов-манипуляторов, которые подходят для решения разных задач, таких как перемещение объектов, заготовок изделий, реагентов и т.п.

Научных работ, затрагивающих тему проблематики управления роботом-манипулятором, сделано относительно мало. Так, например, а работе [2] И. Р. Белоусов описывает методы управления роботом РМ-01. В данной работе проведены эксперименты, в результате которых, были получены приемлемые условия для управления и успешный захват объекта, основанные на использовании трехмерной модели робота в реальном масштабе времени.

Так, в результате проведения конференции по распределенной робототехнике Семинара по распределенной робототехнике и автоматизации, проводимого в рамках Международной конференции по робототехнике IEEE International Conference on Robotics and Automation ICRA'2001 (Сеул, Южная Корея, май, 2001), с использованием виртуальной управляющей среды, рассмотрена проблема удаленного управления роботом, где он управлялся из конференц-зала на расстоянии свыше 10.000 км, где конкретный объект был захвачен манипулятором в реальном масштабе времени. [2]

Такие проведенные сеансы управления роботом через Интернет демонстрировали высокую эффективность разработанных методов.

Таким образом, реализация управляющих программ для удаленного доступа дает возможность оператору непосредственно проводить мониторинг и

управление роботом на больших расстояниях посредством сети Ethernet, вне зависимости от того, где находится компьютер оператора.

1.2 Целевое обследование робота-манипулятора

1.2.1 Анализ устройства промышленного робота

Внешний вид робота-манипулятора RV-2AJ представлен на рис. 1.1

Рисунок 1.1 Внешний вид робота-манипулятора Mitsubishi RV-2AJ

Робот-манипулятор RV-2AJ имеет 5 осей вращения, а также схват, который может быть зафиксирован в двух положениях (либо открыт, либо закрыт). Оси робота обозначаются как J1, J2, J3, J5, J6, их условные обозначения, а также направления осей вращения показаны на рис. 1.2

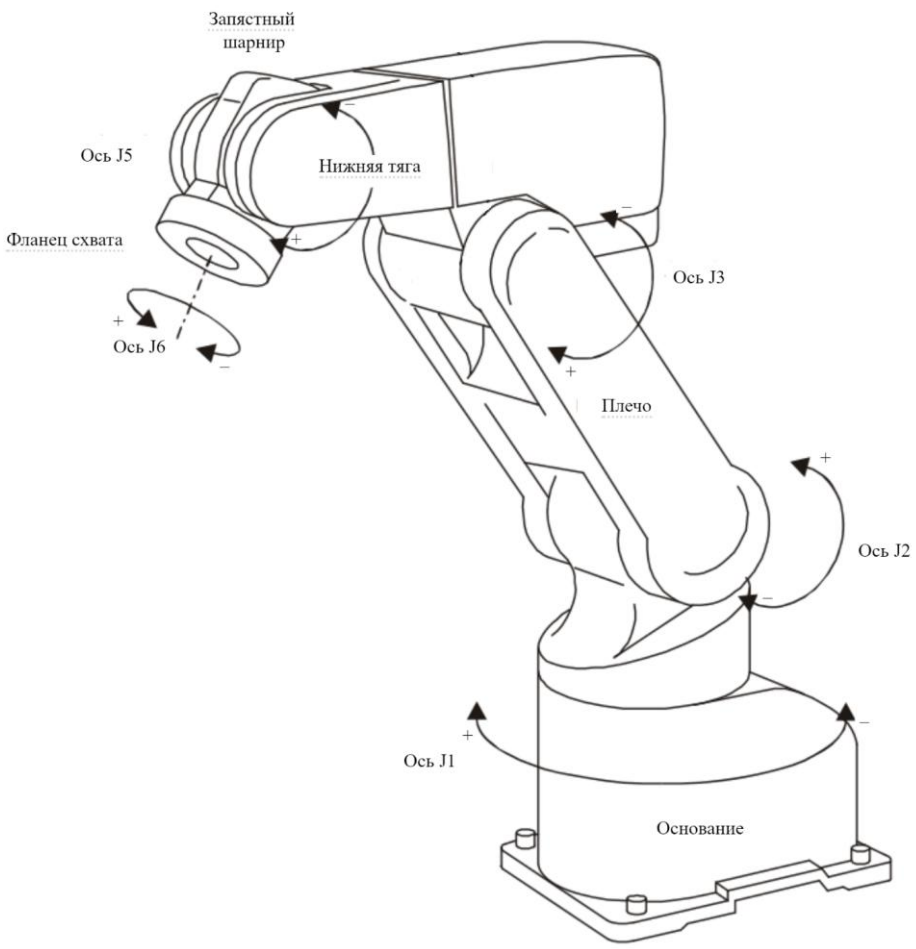

Рисунок 1.2 – Оси вращения робота-манипулятора

Ось J4 является опциональной модификацией данной модели, поэтому в исследуемом манипуляторе в расчет не берется.

Таким образом, робот-манипулятор имеет возможность свободно перемещаться в пространстве с заданной для него скоростью.

1.2.2 Анализ системы координат промышленного робота

Для того, чтобы упростить перемещение схвата робота, его система координат должна отличаться от системы координат осей и должна являться прямоугольной системой координат с центром у основания робота (рис. 1.3).

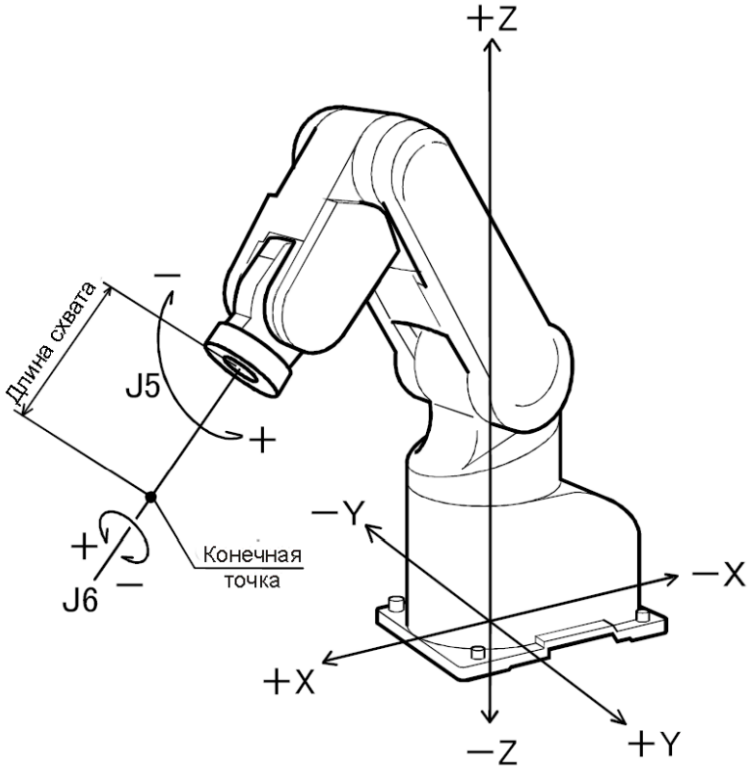

Рисунок 1.3 – Прямоугольная система координат промышленного робота

С помощью данной 3-мерной системы роботу-манипулятору достаточно сообщить координаты точки в пространстве, и робот начнет движение к ней при помощи сервоприводов на своих осях. Еще одним немаловажным фактором для управления роботом является его скорость перемещения от одной точки в пространстве к другой. При проектировании управляющей программы необходимо учитывать эти особенности.

Существуют абсолютные и относительные координаты. Абсолютные координаты отсчитываются каждый раз от начала координат, в нашем случае, от основания самого робота. Относительные же (инкрементальные) основываются на том, что за точку отсчета, каждый раз принимается положение исполнительного органа, которое он занимал перед началом перемещения к следующей точке [3].

Так, каждая из позиций робота, задается при помощи следующих параметров:

1. Координаты в пространстве конечной точки без учета длины схвата [4]. То есть координаты конечной точки, которая является центром из проекции окружности цилиндрического фланца схвата в месте расположения самого схвата. Координаты данной точки непосредственно задаются при помощи 3 значений: X, Y и Z.
2. Вторым параметром выступает угол наклона последнего рабочего органа робота относительно вертикали. В нашем случае, это значение угла поворота оси J5.
3. Третьим параметром выступает угол наклона схвата относительно оси X. А именно, угол самого схвата J6.

Координаты осей X, Y и Z настраиваются в миллиметрах. Координаты осей J5 и J6 корректируются в угловых единицах [5].

Регулировка координаты оси X перемещает руку в радиальном направлении от исходной точки робота. Регулировка координаты оси Y вращает руку вокруг оси J1. Регулировка координаты по оси Z перемещает руку в направлении Z системы координат робота. Регулировка координат осей A, B и C перемещает руку таким же образом, как и при подаче XYZ.

Координаты осей X и Z настраиваются в миллиметрах. Координаты осей Y, A, B и C настраиваются в угловых единицах [5].

1.3 Анализ состояния объекта

1.3.1 Анализ функциональных возможностей

На основании технической документации оборудования [5], оценим возможности отдельного блока управления, сформировав таблицу возможных вариантов толчковой подачи. Доступны следующие пять типов толчковой подачи (табл. 1.1).

Таблица 1.1 – Доступные типы толчковой подачи

Тип	Объяснение
JOINT jog	В этом режиме каждая из осей может регулироваться независимо. Координаты осей от J1 до J6 можно настроить независимо друг от друга. Точное количество осей может отличаться в зависимости от типа робота.
TOOL jog (толчковая подача)	Положение можно отрегулировать вперед/назад, влево/вправо или вверх/вниз относительно направления фланца схвата робота (система координат TOOL). Наконечник движется линейно. Позиция может вращаться вокруг оси X, Y и Z системы координат TOOL фланца схвата нажатием клавиш A, B и C без изменения фактического положения.
XYZ jog	Оси настраиваются линейно относительно системы координат робота. Позиция вращается вокруг осей X, Y и Z робота нажатием клавиш A, B и C, без изменения фактического положения конца руки. Необходимо указать длину инструмента заранее, используя параметр MEXTL.
3-axis XYZ jog	Оси настраиваются линейно относительно системы координат робота. В отличие от подачи XYZ, позиция будет

	такой же, как в случае осей JOINT joint толчковой подачи J5 и J6. Пока положение фланца схвата остается фиксированным, позиция интерполируется X, Y, Z, J5 и J6; т. е. постоянная позиция не поддерживается. Необходимо заранее указать длину инструмента
CYLNDER jog	Используется цилиндрический толчок при перемещении руки в цилиндрическом направление относительно начала робота. Регулировка оси X координата перемещает руку в радиальном направлении от центра робота. Регулировка координаты оси Y перемещает вручную так же, как и в толчковой подаче JOINT вокруг оси J1. Регулировка координаты по оси Z перемещает руку в направлении Z так же, как и при толчковой подаче XYZ. Регулировка координат осей A, B и C вращает рукой так же, как и при толчковой подаче XYZ.

Из табл. 1.1 можем сделать вывод, что робот имеет возможность работать в двух режимах координат: по самим осям робота, либо в декартовой системе координат (XYZ). Однако, из этого же следует, что робот-манипулятор зависит от возможностей конкретного блока управления и не всегда имеет возможность расширения либо модифицирования программ. Поэтому целесообразна разработка такой системы, которая бы позволила в дистанционном режиме управлять данным роботом.

1.3.2 Определение команд, необходимых для управления роботом в прямоугольной системе координат

Робот может перемещаться в системе координат двумя способами.

- линейно (прямолинейно) вдоль направлений ориентации системы координат X, Y, Z;

- вращательно (наклоняясь/ поворачиваясь) вокруг направлений ориентации системы координат: углы A, B и C.

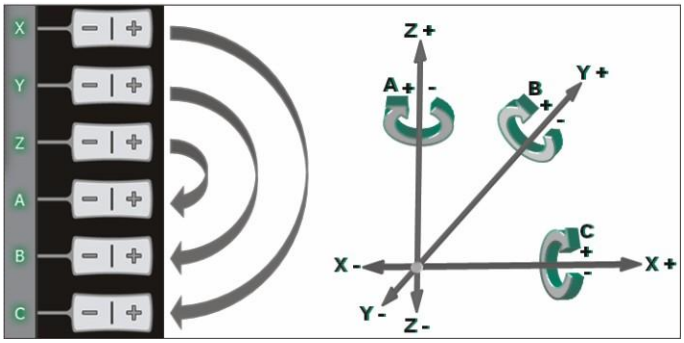

Рисунок 1.4 – Система прямоугольных координат

Параметры и тип рабочих зон промышленного робота предопределяют область пространства, в пределах которой робот может осуществлять при неподвижном основании манипулирование объектами. Все их звенья состоят из шарниров, поэтому их называют шарнирными или антропоморфными. Промышленные роботы с такого типа манипуляторами обладают наибольшей компактностью, хотя и сложны в управлении. [6]

Ангулярная система координат реализуется с помощью трех вращательных кинематических пар при шарнирном соединении звеньев манипулятора, тогда как прямоугольная (декартова) система координат реализуется тремя поступательными кинематическими парами.

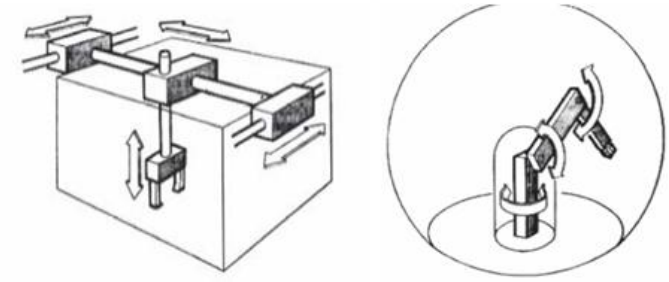

Рисунок 1.4 – Декартова и ангулярная системы координат промышленного робота.

На рис. 1.4 слева отображен принцип работы манипулятора от прямоугольной (декартовой) системы координат, тогда как справа указана угловая (ангулярная) система. [7]

Ангулярная система координат наиболее универсальна, т.к. она обеспечивает обслуживание самого большего объема рабочей зоны и при помощи нее можно строить промышленных роботов, обладающих максимальной эффективностью работы. Однако конструкция манипулятора в таком случае получается довольно сложной, громоздкой и нуждается в сложных системах управления. Недостатком же такой системы является то, что хрупкость манипулятора увеличивается, из-за чего могут возникнуть трудности в обеспечении нужной точности позиционирования.

Прямоугольная система координат. Задавая команду перемещения, система управления сначала рассчитывает отрезок. Исходной точкой данного отрезка является отсчетная точка инструмента. Система управляет всеми осями таким образом, что инструмент проводится по этому отрезку (перенос) или вращается вокруг него (поворот) [9].

Для рассматриваемого робота-манипулятора языком программирования могут выступать MELFA-BASIC IV и MOVEMASTER command. Для передачи команд роботу воспользуемся командами языка MOVEMASTER command.

Список используемых в работе команд и их описание приведено в табл. 1.2.

Таблица 1.2 – Перечень применяемых в работе команд на языке MOVEMASTER command

Тип	**Функция**	**Формат ввода (пример)**
Управление положением	Перемещается к заданной переменной положения с совместной интерполяцией.	MO 100
	Перемещается на указанное расстояние от текущей позиции.	DS 0,10,0

Примечание [#BSA1]: Зачем?

	Перемещает ось на заданную величину от текущего положения.	DJ 5,10
Управление скоростью	Обозначает скорость линейной и круговой интерполяции числовым значением (единица измерения 0,1 мм/с).	SP 15
Управление схватом	Открыть схват (Gripper open)	GO
	Закрыть схват (Gripper close)	GC
Другие команды	Прочитать значение координат текущей позиции	WH
	Считать значение координаты указанного номера позиции.	PR 1
	Сбросить аварийные ошибки	1;1;RSTALRM+
	Сбросить ошибку или номер строки программы.	RS
	Считать текущий номер ошибки или историю ошибок.	ER 1

Примечание [#BSA2]: И эта зачем? Там несколько операторов перемещения в прямоуг.сист . корд.. – их можно сюда включить

Примечание [#BSA3]: И эта зачем?

Таким образом, команды, указанные в табл. 1.2 позволяют управлять роботом-манипулятором в прямоугольной системе координат.

Разберем команды, указанные в табл. 1.2 более подробно.

Оператор DS (Draw Straight) – переместиться на указанное расстояние от текущей позиции

Примечание [#BSA4]: Для остальных операторов нужно оформить подзаголовки аналогично

【 Функция 】

Примечание [#BSA5]: Уберите нестандартные символы
Я не говорил скопировать один в один разделы мануала по роботу. Нужно было сделать краткое описание команд и операторов своими словами.

Перемещает фланец схвата в позицию, удаленную от текущей позиции на расстояние, указанное в направлениях X,Y и Z. (Линейная интерполяция)

【 Формат ввода 】

DS <расстояние перемещения в X>, <расстояние перемещения в Y>, <расстояние перемещения в Z>

【 Термин 】

<Расстояние перемещения в X> Требуется указать величину, которую необходимо переместить в направлении X из текущей позиции.

<Расстояние перемещения в Y> Требуется указать величину, которую необходимо переместить в направлении Y от текущей позиции.

<Расстояние перемещения в Z> Требуется указать величину, которую необходимо переместить в направлении Z от текущей позиции.

【 Объяснение 】

1) Наименьшее входное приращение для расстояния составляет 0,01 мм

2) Положение руки, включая состояние открытия/закрытия захвата, остается неизменным до и после перемещения.

3) Ошибка возникает до или во время движения, если пункт назначения или траектория перемещения превышает рабочее пространство робота.

В частности, шарнир качения имеет тенденцию превышать рабочее пространство, поскольку он управляется так, чтобы оставаться в одной и той же ориентации во время перемещения.

4) Скорость перемещения во время линейной интерполяции определяется командами SP или SD.

5) Наконечник для руки определяется длиной инструмента.

【 Пример программы 】 (команда MOVEMASTER)

10 DS 100,0,0 ; Перемещается в направлении оси X на 100 мм.

DJ (Draw Joint) – переместить ось на заданную величину от текущего положения.

【 Функция 】

Поворачивает указанное соединение на указанный угол от текущего положения. (Совместная интерполяция)

【 Формат ввода 】

DJ <номер соединения>, <угол поворота>

【 Термин 】

<Номер соединения> Требуется указать номер соединения, которое необходимо переместить.

1) 6-осевой тип J1… J6, J7 (дополнительная ось 1), J8 (дополнительная ось 2)

2) 5-осевой тип. J1… J3, J4… J6, J7 (дополнительная ось 1), J8 (дополнительная ось 2)

<Угол поворота> Требуется указать величину соединения, которое необходимо переместить.

【 Объяснение 】

1) Наименьшее приращение каждой оси составляет 0,01 для серии RV-1A (мм или градус). Например, требуется указать 20,01 для 20,01 мм для RV-1A)

2) Состояние открытия/закрытия руки не меняется до и после движения. Ошибка возникает перед совместным движением, если какой-либо угол поворота превышает рабочее пространство робота.

【 Пример программы 】 (команда MOVEMASTER)

10 MO 1; Перемещается в положение 1.

20 DJ 1,10 ; Поворачивает ось J1 на 10 градусов в положительном направлении.

Примечание [#BSA6]: Это угловое перемещение
Как оно здесь используется?

SP (Speed) – рабочая скорость

【 Функция 】

Задает рабочую скорость, время ускорения или замедления и настройку непрерывной траектории.

【 Формат ввода 】

SP <уровень скорости> [, [<H/L >] [, <Настройка CNT>]]

【 Термин 】

<Уровень скорости> Установите скорость движения.

0 ≤ уровень скорости ≤ 30

<H/L> Установите уровень ускорения/замедления. (По умолчанию используется значение H.)

H: Высокое ускорение/замедление (макс. 100%)

L: Низкое ускорение/замедление (макс. 50%)

<Настройка CNT> Требуется указать состояние включения/выключения режима непрерывного пути. (По умолчанию значение отключено.)

0: Отключить 1: Включить

【 Объяснение 】

1) Устанавливает рабочую скорость в 31 шагов и время ускорения/замедления при запуске и остановке на 2 уровнях.

2) Уровень скорости задается как отношение к максимальным оборотам в минуту каждого шарнира для совместной интерполяции и как отношение к максимальной скорости фланца схвата для линейной интерполяции. Начальное значение равно 30.

3) Время ускорения/замедления может быть выбрано из числа H или L. Время разгона является максимальным временем, необходимым роботу для достижения максимальной скорости. Следовательно, когда скорость движения не достигает максимальной скорости, фактическое время ускорения становится меньше заданного значения.

4) Время команды ускорения/замедления может быть определено в двух этапах H и L. В любом случае скорость соответствует максимальной скорости ускорения/замедления для совместной, линейной и круговой интерполяции и обозначает ускорение во время запуска. (Это также относится и к замедлению.) Таким образом, если скорость движения не достигает при максимальной скорости время разгона/замедления будет меньше установленного значения.

5) Для линейной интерполяции фланец схвата, определяемый командой инструмента, перемещается с постоянной скоростью. В этом случае ошибка может возникнуть из-за превышения максимальной скорости любого из соединений. Если движение позиционного угла (5-осевой тип: угол A, B) больше, чем расстояние перемещения (X, Y, Z), робот перемещается относительно угловой скорости положения.

Команда SD позволяет определять скорость с меньшими приращениями.

6) Значение CNT по умолчанию недействительно в качестве настройки по умолчанию.

7) Включив настройку CNT, робот движется непрерывно без ускорения и замедления до тех пор, пока SD или команда SP отключает настройку CNT. (движение по траектории). Однако робот ускоряется и замедляется в начальной и конечной точках, а также при выполнении таймера или входной команды во время движения по траектории.

8) Скорость и ускорение/замедление, установленные один раз, действительны до тех пор, пока не будут установлены снова. Если значение H/L опущено, предыдущее значение настройки будет оставаться неизменным (если выполняется команда SD, это значение настройки будет действительным).

(9) Настройка возвращается к значению по умолчанию при выполнении команды ED или сбросе программы.

GO (Grip Open) – разомкнуть(открыть) схват

【 Функция 】

Разжимает схват руки.

【 Формат ввода 】

GO [<номер руки>]

【 Термин 】

<Номер руки> Требуется указать номер руки, открывающий схват.

Рука 1 (По умолчанию)

【 Объяснение 】

1) Эта команда приводит в действие электромагнитный клапан и открывает руку (или освобождает заготовку).

Выходным сигналом общего назначения также можно управлять с помощью команды OB для открытия руки.

2) При разжимании руки и отпускании заготовки может потребоваться некоторое время, чтобы движение установилось и рука начала двигаться. Таким образом, в некоторых случаях эта команда должна быть предоставлена до и после этой команды, чтобы установить таймер задержки.

【 Пример программы 】 (команда MOVEMASTER)

10 MO 10,C ; перемещается в положение 10 с закрытой рукой

20 TI 5 ; устанавливает таймер на 0,5 секунды.

30 GO ; разжимает руку, чтобы освободить заготовку..

GC (Grip Close)

【 Функция 】

Сжимает схват руки.

【 Формат ввода 】

GC [<номер руки>]

<Номер руки> Требуется указать номер руки, открывающий схват.

Рука 1 (По умолчанию)

【 Объяснение 】

1) Эта команда приводит в действие электромагнитный клапан и закрывает руку (или поднимает заготовку).

Выходным сигналом общего назначения также можно управлять с помощью команды OB для закрытия руки.

2) При закрытии руки и захвате заготовки может потребоваться некоторое время, чтобы движение установилось и рука начала двигаться. Таким образом, в некоторых случаях эта команда должна быть предоставлена до и после этой команды, чтобы установить таймер задержки.

【 Пример программы 】 (команда MOVEMASTER)

10 MO 10,O ; перемещается в положение 10 с открытой рукой.

20 TI 5 ; устанавливает таймер на 0,5 секунды.

30 GC ; закрывает руку для захвата заготовки.

Робот так же поддерживает возможность движения по круговой интерполяции и непрерывное движение.

Для робота RV-2AJ использование этих команд ограничено из-за пяти степеней свободы. [8].

WH (Where) – прочитать значение координат текущей позиции

【 Функция 】

Считывает координаты текущей позиции и состояние открытия или закрытия руки. (С использованием RS-232C)

【 Формат ввода 】

WH

【 Объяснение 】

1) Вызывает вывод координат текущего положения фланца схвата, определяемого длиной инструмента, и состояния открытия или закрытия руки через порт RS-232C.

2) Формат вывода имеет кодировку ASCII, как указано ниже. Наименьшее приращение выходного сигнала составляет 0,01.

Формат вывода:

5-осевой тип: Значение координат X, Y, Z, угол поворота A, B, дополнительная ось 1, дополнительная ось 2, R/L, A/B, O/C

【 Пример программы 】 (команда MOVEMASTER)

10 OPEN ” COM1:E83” AS #1 ; устанавливает связь RS-232C с ПК

20 PRINT #1,” WH” ; передает команду "WH".

40 LINE INPUT #1,A$; сохраняет полученные данные в A$

50 PRINT "Current coordinates are " : A $; отображает содержимое A$ на экране.

60 ED ; Ends. конец

PR (Position Read) – считать значение координаты указанного номера позиции

【 Функция 】

Считывает координаты указанной позиции и состояние открытия/закрытия руки (с использованием RS-232C).

【 Формат ввода 】

PR [<номер позиции>]

【 Термин 】

<Номер позиции> Укажите номер позиции, который необходимо прочитать.

0 ≤ номер позиции ≤ 999

(Если опущено, действителен номер текущей позиции.)

【 Объяснение 】

1) Выводит координаты заданного положения и состояние открытия/закрытия рукоятки через порт RS-232C.

Если номер позиции опущен или равен 0, выводится только номер текущей позиции.

2) Данные кодируются в формате ASCII следующим образом; Наименьшее приращение составляет 0,01 мм или 0,01 градуса.

Формат вывода

5-осевой тип: Значение координат X, Y, Z, градус угла поворота A, B, R / L, A / B, O / C

3) Поскольку конечным значением выходных данных является возврат каретки (шестнадцатеричный 0D), при получении сообщения персональным компьютером необходимо обрабатывать последовательные строки данных с точностью до шестнадцатеричного 0D.

4) Если необходимо считать позицию, которая не используется в программе и еще не определена, с каждой координатой выводится "ноль".

5-осевой тип: 0,0,0,0,0

Если необходимо считать позицию, которая уже используется в программе, но еще не определена, то для каждой координаты выводится значение "0.00".

5-осевой тип : 0.00,0.00,0.00,0.00,0.00

5) Если указать "ноль" для номера позиции или опустить его, вы можете определить номер текущей позиции при возникновении ошибки при выполнении команды перемещения.

Данный робот имеет следующие конструктивные параметры, мм:

$_{S1}$=300, a_2=250, a_3=160, S_{5M}=72 [9].

Выводы по разделу. Проведение предпроектного обследования робота-манипулятора позволило проанализировать состояния проблемы по информационным источникам, провести анализ функциональных возможностей робота, проанализировать его устройство, определить команды, необходимые для управления роботом в прямоугольной системе координат. По результатам работы данного раздела получена аналитическая модель, основанная на физической (архитектурной) модели робота.

2. КОНЦЕПТУАЛЬНОЕ ПРОЕКТИРОВАНИЕ СИСТЕМЫ

2.1 Выбор общесистемной методологии проектирования

При проектировании информационных систем есть 2 подхода или метода: объектно-ориентированный и структурный анализы.

Структурный подход при разработке ИС заключается в том, что система разделяется на функциональные подсистемы, которые делятся на подфункции, которые в свою очередь могут разделяться на задачи и т.д. Этот процесс продолжается до достижения конкретных процедур. В данном подходе сохраняется связь между всеми компонентами ИС[10].

С помощью же объектно-ориентированных методов можно построить (описание) модель предметной области в качестве совокупности объектов, которые являются сущностями и объединяют в себе сами данные и методы их обработки (процедуры и методы) [11]. Крайним результатом этого подхода должно стать множество классов объектов вместе с соединенными методами обработки самих атрибутов. В соответствии с этим, ИС, основанная на таком подходе, последовательно расширяется в направлении от представления общей модели функциональности ИС до динамической модели взаимодействия между объектами, на основе которой могут быть построены классы объектов в конкретной программной среде. Учитывая эти особенности целесообразно выбрать объектно-ориентированный подход при проектировании ИС. На сегодняшний момент для построения модели на основе объектно-ориентированного подхода широко используется унифицированный язык моделирования UML, который фактичекски является стандартом для реализации данного подхода.

2.2 Выбор CASE-средства проектирования

Существует довольно много CASE-инструментов проектирования и моделирования систем и баз данных, однако наиболее полную модель процессов на разных уровнях дает язык UML.

При помощи диаграмм UML можно отобразить практически все процессы и порядок их взаимодействия между ними. UML является одним из общепринятых языков для визуализации моделирования систем [12].

Система объектно-ориентированных моделей в соответствии с нотациями UML включает в себя следующие диаграммы:

- диаграмму прецедентов использования (Use-case diagram), которая отображает функциональность ИС в виде совокупности выполняющихся последовательностей транзакций;
- диаграмму классов объектов (Class diagram), которая отображает структуру совокупности взаимосвязанных классов объектов аналогично ER-диаграмме функционально - ориентированного подхода;
- диаграммы состояний (Statechart diagram), каждая из которых, отображает динамику состояний объектов одного класса и связанных с ними событий;
- диаграммы взаимодействия объектов (Interaction diagram) где каждая из них отображает динамическое взаимодействие объектов в рамках одного прецедента использования;
- диаграммы деятельностей (Activity diagram), которые отображают потоки работ во взаимосвязанных прецедентах использования (могут декомпозироваться на более детальные диаграммы);
- диаграммы пакетов (Package diagram), которые отображают распределение объектов по функциональным или обеспечивающим подсистемам (могут декомпозироваться на более детальные диаграммы);

- диаграмму компонентов (Component diagram), которая отображает физические модули программного кода;
- диаграмму размещения (Deployment diagram), которая отображает распределение объектов по узлам вычислительной сети.

В данной работе в качестве инструмента проектирования моделирования системы выбран программный инструмент моделирования draw.io.

Данная программная среда имеет свободную лицензию и доступна для установки с официального сайта.

2.3 Разработка UML диаграмм

2.3.1 Разработка Use-case диаграммы

При проектировании важно учитывать, что программой может пользоваться и в дальнейшем разрабатывать не один разработчик, а множество людей. Поэтому целесообразно разработать диаграмму вариантов использования. Данный вид диаграммы описывает, какой именно функционал доступен для каждого вида пользователя. Т.к. в нашем случае доступ к функциям программы не разграничивается по ролям, то будем описывать целевого пользователя – оператора.

В результате проектирования была разработана следующая диаграмма вариантов использования (рис 2.1)

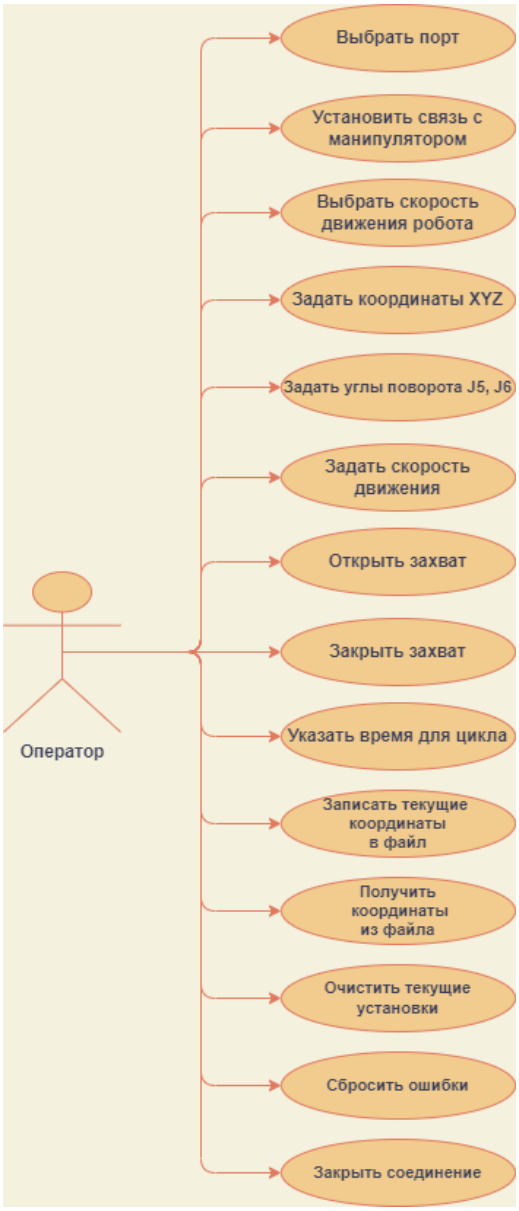

Рисунок 2.1 – Use-case диаграмма вариантов использования

2.3.2 Разработка диаграммы компонентов

Диаграммы компонентов применяются для визуальной интерпретации компонентов системы и их взаимосвязей и зависимостей между собой. [13]

В данной схеме, компонентами могут выступать: база данных или же аппаратные компоненты, такие как микросхема или устройство, пользовательский интерфейс и др. Диаграмма компонентов представлена на рис. 2.2.

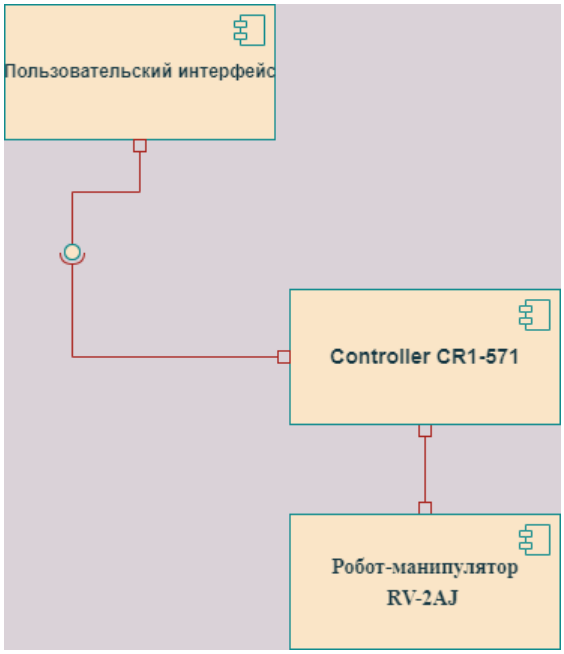

Рисунок 2.2 – UML Диаграмма компонентов

2.4 Разработка функциональной структуры системы

В качестве инструмента для проектирования функциональной структуры применим нотацию IDEF0. Данная модель представляет из себя набор блоков, которые являются как бы «черными ящиками». Они имеют свои входы и

выходы, управления и механизмов, которые детализируются (декомпозируются) до необходимого уровня. [14]

Однако, у нее есть и недостатки, так, например, она плохо отражает реакцию участников данного процесса на события из внешней среды. Тем самым нельзя оценить риски, которые связанны с изменениями во внешней среде. Поэтому, нотация IDEF0 подходит больше для описания именно технологических процессов, у которых исключено влияние окружающей среды, т.к. все операции выполняются по заготовленному плану. В расчет не берутся поломки оборудования. Из этого следует, что возникает либо задержка, либо отмена процесса на некотором этапе, тогда как при возникновении исключительных ситуаций в бизнес-среде в большинстве случаев требуются именно действия по откату. Именно поэтому нотация IDEF0 кажется более приемлемой для описания процессов технологических. Верхний уровень системы отображен на схеме (рис. 2.3)

Так, в данной нотации отражены зависимости каждого компонента системы от различных факторов, наличия необходимого оборудования, литературы и т.д. Так, например, для установления связи между компьютером и роботом-манипулятором необходимо наличия питания на обоих устройствах, целостный помехоустойчивый кабель RS-232 для обеспечения бесперебойной связи между компьютером и контроллером, а также кабели 1A-LCBL(P)-1 и 1E-LCBL(S)-N для связи между контроллером и манипулятором. А в роли «Механизма» для установления связи выступает оператор. Который и посылает управляющие команды манипулятору посредством программного интерфейса (рис. 2.3).

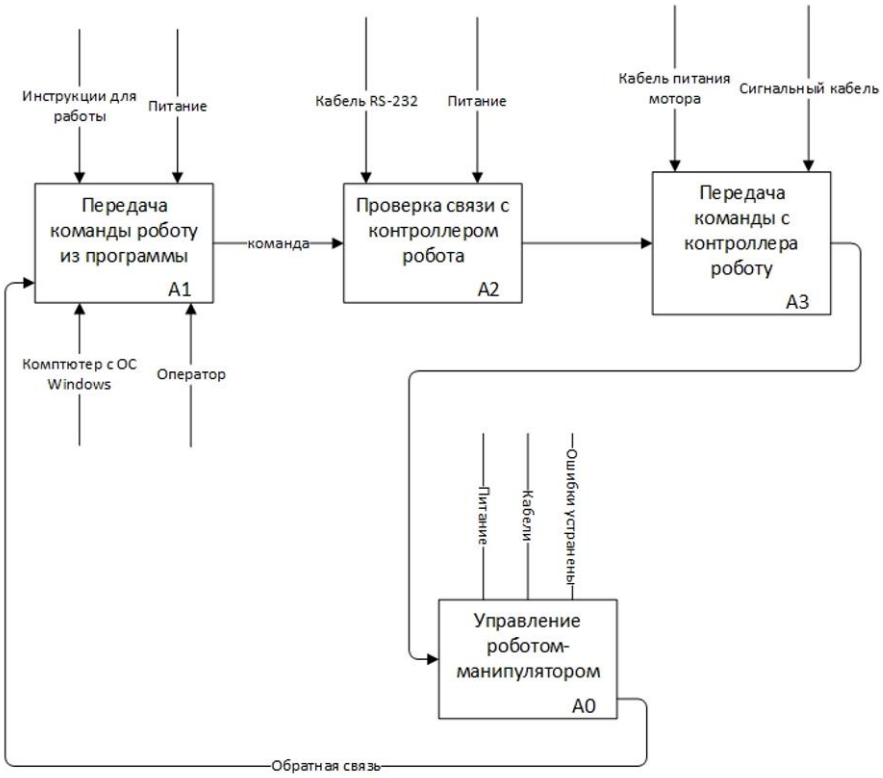

Рисунок 2.3 – IDEF0 нотация

Исходя из построенных диаграмм, можно сформировать соответствующий вывод по разделу о том, что для качественной разработки ПО необходимо, прежде всего, правильно выбрать подход к построению такой системы, систематизировав полученные знания и опыт в виде диаграмм (UML, IDFE и др.), что упростит разработку программного обеспечения в дальнейшем.

3. ТЕХНИЧЕСКОЕ ЗАДАНИЕ

Введение

Наименование программы

Наименование программы – «Robot Melfa Control».

Краткая характеристика области применения

Программный комплекс служит для управления роботом-рукой Mitsubishi RV-2AJ в пространстве. У робота предусмотрен свой язык команд, принимая которые, он осуществляет движения. Связь с роботом осуществляется посредством RS-232.

Основания для разработки.

Основанием для разработки являются задачи, поставленные на кафедре для проектирования ВКР.

Исполнитель обязуется разработать программную систему на компьютере с операционной системой Windows, предоставить результаты работы, исходные коды, результаты тестирования и документацию к разработанной системе не позднее 01.06.2022.

Наименование темы разработки – «Программное средство удаленного управления роботом-манипулятором в прямоугольной системе координат».

Условное обозначение темы разработки (шифр темы) – «Robot1».

Назначение разработки

Программа будет использоваться для удаленного управления роботом-манипулятором, задавая для него необходимые установки. [15]

Функциональное назначение

Для сотрудника, который может пользоваться данной программой, программа предоставляет возможность управления и просмотра координат текущего положения робота, запись и считывание координат из базы данных.

Эксплуатационное назначение

Программа может эксплуатироваться непосредственно в производственном помещении, связь между компьютером и контроллером робота осуществляется через RS-232 соединение. Права доступа к роботу-манипулятору не учитываются.

Требования к программе или программному изделию

Разработка предварительных проектных решений по системе и ее частям

Требования к составу выполняемых функций [16]

После запуска программы, на экране отображается форма с кнопками управления роботом с идентичным содержанием которой, отображено на рис 4.1

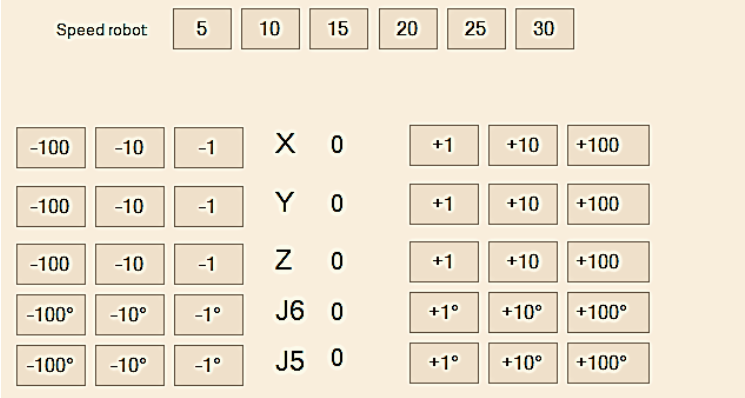

Рисунок 3.1 – Интерфейс управляющих команд

Для пользователя роботом-симулятором программа предоставляет следующие возможности:

- выбрать соответствующий порт;
- выбрать координаты XYZ и углы наклона последнего рабочего органа робота относительно вертикали(J5), а также угол наклона схвата относительно оси X(J6).
- задать скорость движения робота
- открыть/закрыть захват

- повторять движения в течении заданного времени
- записать введенные управляющие команды с координатами в файл.
- прочитать управляющие команды с координатами из файла
- сбросить ошибки
- просмотреть логирование

Примерный вид окна просмотра результатов и состав выполняемых функций показан на рисунке 4.2.

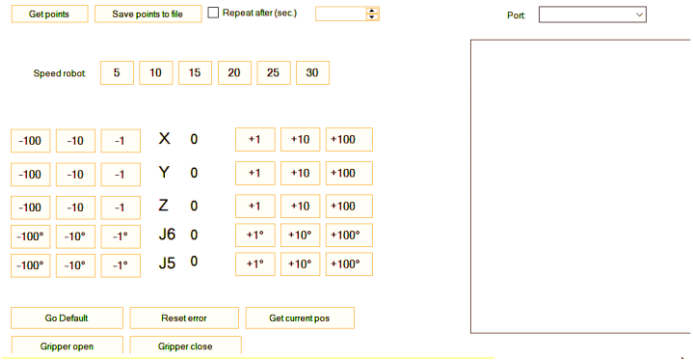

Рисунок 3.2 – Примерный интерфейс главного окна программы

Вверху окна 2 управляющих кнопки для работы с файлом. *Get points* – считать значение координаты указанного номера позиции. Save points to file – записать текущие координаты в файл. Также рядом установлен компонент numericUpDown, в котором можно выбрать время *t*(сек) для задания цикла. Цикл будет повторяться через каждые *t* сек, если checkBox предварительно установлен в True. Правее от них расположен компонент ComboBoxList, в который подгружается список доступных портов компьютера.

Немного ниже располагается строка задания скорости работы робота, от 5 до 30 с шагом в 5 пунктов. Если нажать на соответствующую кнопку выбора скорости, то остальные должны стать неактивными.

Далее на панели можно выбрать необходимое число по каждому из 5 параметров. Результат будет суммироваться по каждому из них.

Примечание [#BSA8]: В чем суть?

Примечание [#BSA9]: НЕТ В ЗАДАНИИ РАБОТ С СОХРАНЯЕМЫМИ В КОНТРОЛЛЕР ПОЗИЦИЯМИ!
Нам не нужно их сохранять и получать!

Примечание [#BSA10]: Не нужна эта кнопки!

Примечание [#BSA11]: И эта не нужна!

Примечание [#BSA12]: Что это за цикл? Зачем этот функционал здесь?
И что за чекбокс «Repeat after(sec.)»?

Внизу формы располагаются 5 управляющих кнопок.

- *Go Default* – сбросит текущие набранные настройки
- *Reset Error* – отправит команду *RSTALRM* манипулятору и сбросит ошибки
- *Get Curent pos* – отправит команду *WH* манипулятору, который вернет координаты текущей позиции.
- *Gripper open* – откроет схват
- *Gripper close* – закроет схват
- Справа расположен компонент *LogList* в который дозаписываются координаты текущей позиции манипулятора.

Требования к организации входных и выходных данных

Требуется обеспечить сохранение данных о позициях робота в файл. Выходные данные должны обеспечивать возможность отправки команд для прямолинейного перемещения робота-манипулятора в прямоугольной системе координат

Примечание [#BSA13]: НЕТ В ЗАДАНИИ РАБОТ С СОХРАНЯЕМЫМИ В КОНТРОЛЛЕР ПОЗИЦИЯМИ!
Нам не нужно их сохранять и получать!

Требования к временным характеристикам

После добавления новых данных, новая информация о координатах должна отображаться не позднее, чем через 3 секунды.

Требования к надежности

Вероятность безотказной работы системы должна составлять не менее 99.3% при условии исправности сети.

Требования к обеспечению надежного (устойчивого) функционирования программы

Надежное (устойчивое) функционирование программы должно быть обеспечено выполнением заказчиком совокупности организационно-технических мероприятий:

- использованием лицензионного программного обеспечения;
- организацией бесперебойного питания технических средств;

- регулярным выполнением рекомендаций Министерства труда и социального развития РФ, изложенных в Постановлении от 23 июля 1998 г. «Об утверждении межотраслевых типовых норм времени на работы по сервисному обслуживанию ПЭВМ и оргтехники и сопровождению программных средств»;
- регулярным выполнением требований ГОСТ 51188-98. Защита информации. Испытания программных средств на наличие компьютерных вирусов. [17]

Время восстановления после отказа

Время восстановления после отказа, которое может быть вызвано сбоем электропитания ТС (иными внешними факторами), не фатальным сбоем (не крахом) операционной системы, не должно превышать 10 минут при условии соблюдения условий эксплуатации технических и программных средств.

Время восстановления после отказа, вызванного неисправностью ТСА, сбоем операционной системы, не должно превышать времени, которое требуется для устранения неисправностей технических средств и их переустановки.

Отказы из-за некорректных действий оператора

Отказы программы возможны вследствие некорректных действий оператора (пользователя) при взаимодействии с программой. Для того, чтобы избежать возникновения отказов программы по данной причине следует ограничить отклик манипулятора, до появления новой команды, а также ограничить максимальное число (доступных для манипулятора) координат, введенных пользователем.

Условия эксплуатации

Программа запускается на компьютере оператора. База данных находится на этом же компьютере.

Окно программы должно иметь возможность открываться на весь экран, а также должна быть возможность свернуть, закрыть приложение или изменить

размер окна. Запуск программы может осуществляться в любой момент на запущенной операционной системе Windows XP и выше.

Климатические условия эксплуатации

Данные специальные условия не требуются.

Требования к видам обслуживания

Программа не требует проведения обслуживания.

Требования к численности и квалификации персонала

Программа не требует установки. Может запускаться в портативной версии исполнительным .exe файлом.

Оператор(пользователь) должен иметь среднетехническое или высшее профильное образование, а также обладать навыками работы с графическим пользовательским интерфейсом операционной системы и данной программы.

Требования к составу и параметрам технических средств

Состав технических средств:

Компьютер оператора, включающий в себя:

процессор x86 с тактовой частотой, не менее 800 Гц;

оперативную память объемом, не менее 512 Мб;

монитор, видеокарту, клавиатуру, мышь, диск, объемом не менее 5 Гб.

Робот-манипулятор RV-2AJ, включающий в себя:

контроллер CR1-571;

кабель RS-232

Требования к информационной и программной совместимости

Приложение обменивается данными с контроллером манипулятора посредством протокола RS-232, соединенным соответствующим кабелем. Должно быть исключено появление помех посредством кабеля.

Требование к маркировке и упаковке

Специальных требований к маркировке не предъявляется.

Требования к транспортированию и хранению

Специальных требований к маркировке не предъявляется.

Специальные требования

Программа должна обеспечивать взаимодействие с оператором посредством графического пользовательского интерфейса, разработанного согласно требованиям.

Технико-экономические показатели

Программа «Robot Melfa Control» пригодна для удаленного управления роботом RV-2AJ, задействованном в производственном процессе.

Функциональность программы совпадает с аналогами.

В случае сравнительно небольшой стоимости программы, потребность в ней может быть весьма высокой — управляющие программы требуют постоянных модификаций. Экономический эффект при этом может быть обеспечен за счет платной установки системы.

По результатам работы сформированного технического задания будет разрабатываться управляющая программа, которая будет учитывать все требования заказчика к предлагаемой программе, и, на основании этого документа, будет разрабатываться соответствующий программный продукт.

4. СТРУКТУРНОЕ (ТЕХНИЧЕСКОЕ ПРОЕКТИРОВАНИЕ) УПРАВЛЕНИЯ РОБОТОМ

4.1 Разработка функциональной и структурной схемы

Разработаем общую функциональную схему для данного промышленного робота. Она включает в себя графический интерфейс программы, а также саму управляющую программу, для наглядности, эти компоненты были функционально разделены. Посредством графического интерфейса реализуется взаимодействие с управляющей программой. Далее информация посредством связи открытого COM-порта передается через интерфейс RS-232 напрямую в контроллер робота, который в свою очередь, обрабатывает принятые команды и посылает соответствующий управляющий сигнал на приводы. Механическая система манипулятора осуществляет взаимодействие с внешней средой.

Данная схема представлена на рис. 4.1

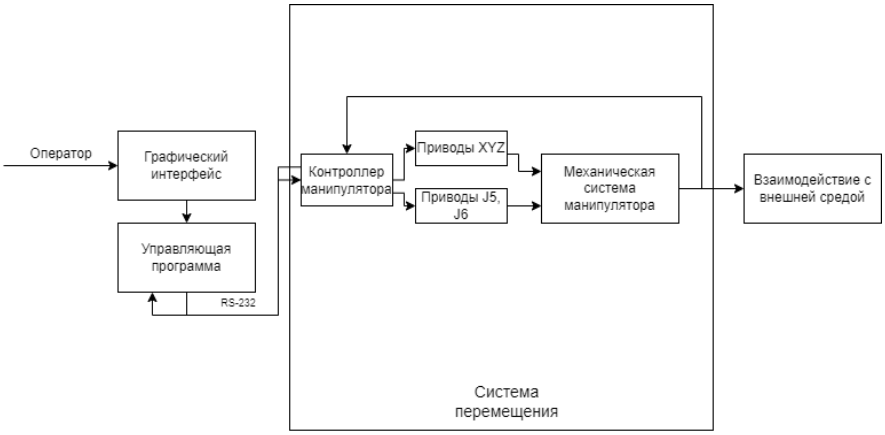

Рисунок 4.1 Функциональная схема управления роботом-манипулятором

Структурная схема позволяет наглядно продемонстрировать принцип работы системы и ее отдельных компонентов.

Структурная схема промышленного робота представлена на рис. 4.2.

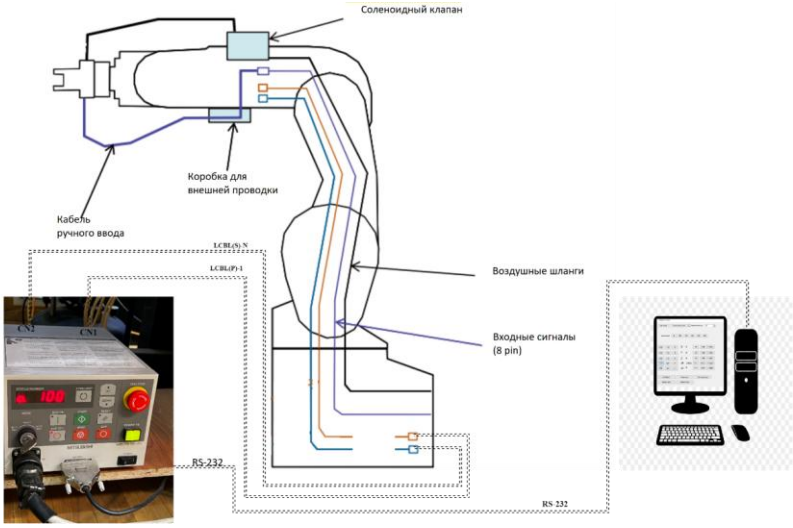

Рисунок 4.2 – Структурная схема управления промышленным роботом

На рисунке 4.2 отображена структурная схема согласования робота-манипулятора посредством блока CR1-571 с управляющей программой на рабочей станции, под управлением ОС Windows.

Исполнительное устройство робота выполняет все его двигательные функции. Исполнительное устройство состоит из манипулятора, и, в общем случае, устройства перемещения промышленного робота. Манипулятор состоит из несущих конструкций, редукторных приводов, передаточных и исполнительных механизмов. Механическая рука позволяет осуществлять ориентирующие и транспортирующие движения.

На данной структурной схеме отображены связующие компоненты АСУ и их взаимосвязь.

4.2 Разработка блок-схемы основных алгоритмов работы программы

Для того, чтобы описать последовательность работы программы, осуществления управления, разработаем блок-схему и опишем ее работу.

Вначале загрузки среды, загружается основная форма программы. И далее система ожидает действия пользователя, поэтому алгоритм работы программы показан как завершенный.

Сначала проверяется, установлено ли уже соединение с роботом. Если нет, то устанавливаем его. Ожидаем команду от пользователя. Как только команда поступает, в обработчике событий данной кнопки снова проверяется, существует ли с роботом соединение. И если такого соединения нет, то срабатывает соответствующий метод обработки ошибок и в лог выводится сообщение о типе ошибке. Иначе, проверяем, корректный ли ответ робота? Если ответ некорректный, то отправляем в обработчик ошибок ответ от робота и выводим результат после обработчика ошибок в лог. Если ответ корректный, то выводим в лог текущую позицию робота (с учетом новой команды).

Блок схема работы при загрузке системы представлена на рисунке 4.5

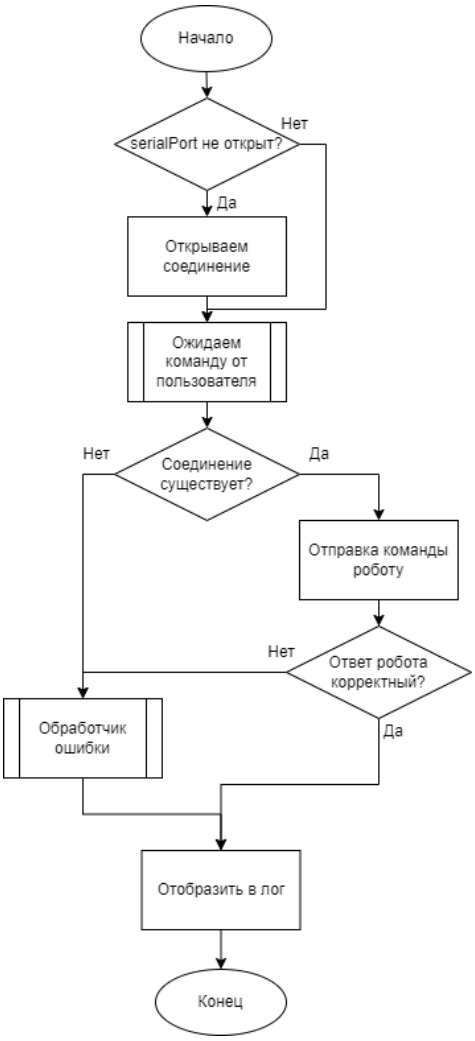

Рисунок 4.5 – Общий алгоритм работы программы ожидания команды от пользователя

Выводы по разделу можно сформировать следующие. В ходе работы разработана функциональная и структурная схемы, определен метод и протокол подключения персонального компьютера к контроллеру робота, разработан алгоритм работы программы при загрузке системы, а также алгоритм работы одного из методов программы.

5. РАБОЧЕЕ ПРОЕКТИРОВАНИЕ

5.1 Разработка интерфейса программы

Целью данного проектирования является применение полученных навыков для практической разработки ПО. Для создания оптимального приложения, которое удовлетворило бы потребностям (указанных в ТЗ) в удаленном управлении манипулятором. Для создания визуального программного интерфейса воспользуемся языком программирования C# (Windows Forms). Данное решение обосновывается тем, что среда разработки Visual Studio имеет удобные графические инструменты для работы с оконными приложениями на языке C#.

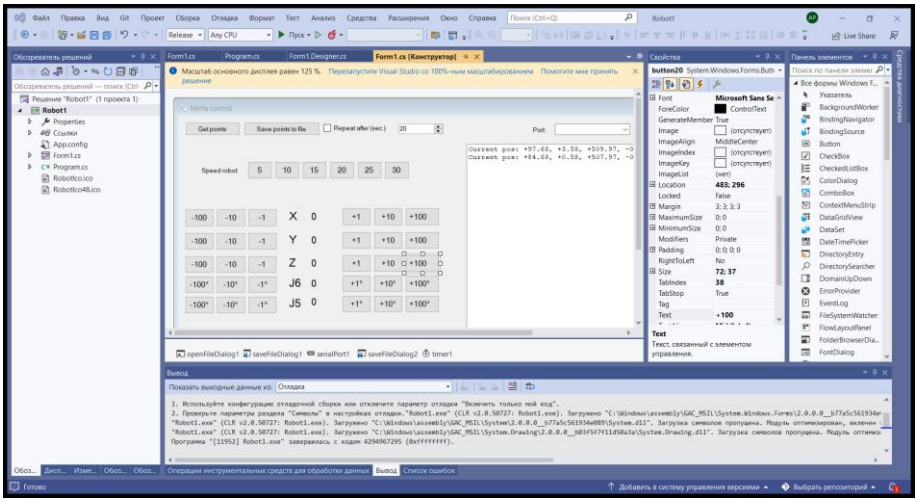

Рисунок 5.1. Процесс разработки графического окна приложения

Графический интерфейс разработанного приложения показан на рис. 5.2.

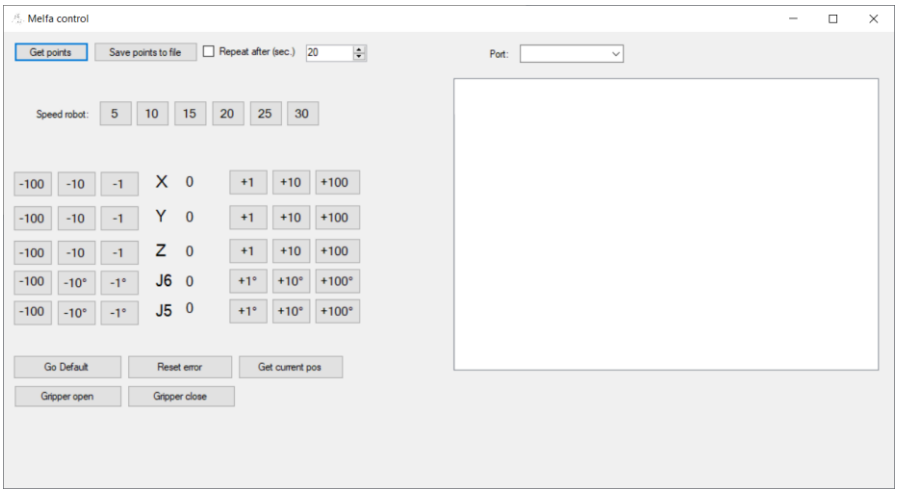

Рисунок 5.2 – Графический интерфейс стартового окна программы

После выбора порта и нажатия любой из управляющих копок, происходит проверка на существование подключения, и если проверка прошла успешно, то посылается соответствующая команда управления.

При получении текущей позиции робота (*Get current pos*), соответствующие данные текущего положения манипулятора заносятся в логи (рис. 5.2).

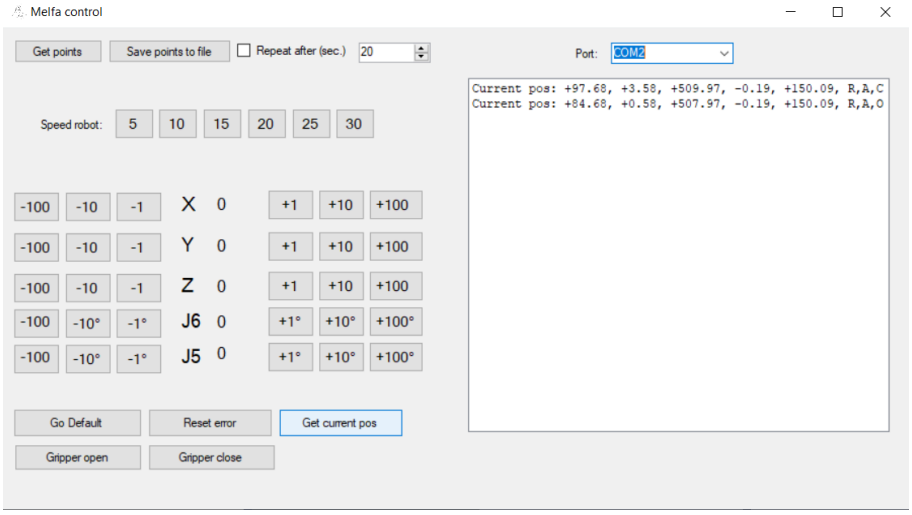

Рисунок 5.2 – Запись новой позиции манипулятора в лог

После задания новых координат в пространстве по осям X,Y,Z и выбор углов поворота, необходимо также указать скорость движения, выбрав соотвествующую из строки управления, после чего остальные станут неактивными. Этот процес показан на рис. 5.3

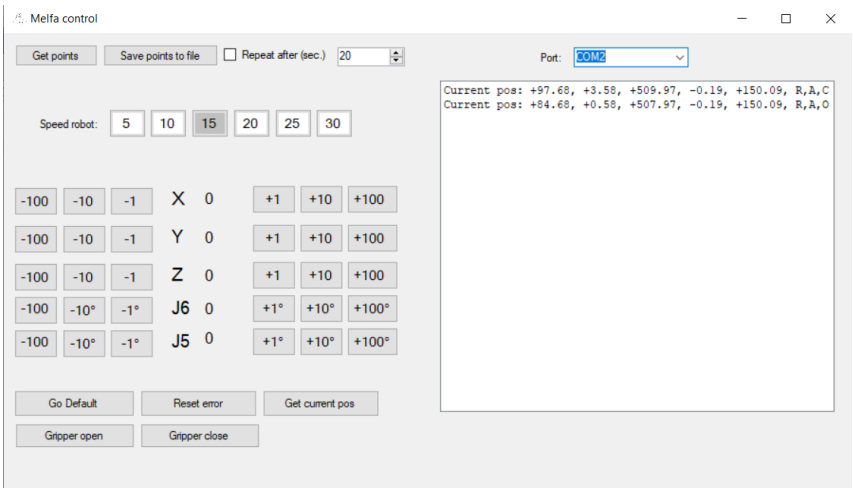

Рисунок 5.3 – Процесс выбора координат и скорости для управления

5.1 Разработка программного кода

При задании новой команды роботу-манипулятору, сначала проверяем, есть ли между ним и ПК подключение, во избежание ошибок. Рассмотрим несколько программных функций. Так рассмотрим обработчик ошибок.

```
private void serialPort1_DataReceived(object sender,
System.IO.Ports.SerialDataReceivedEventArgs e)
{
   string res = serialPort1.ReadTo(End.ToString());
      if (res != "")
         {
         if(GetWhere)
         {
            GetWhere = false;
            AddSerialMonitorLine("Current pos: " + res);
            return;
         }

         if (GetError)
```

```
          {
             GetError = false;
             switch (res)
          {
          case "0": {
             AddSerialMonitorLine("Ok"); break;
                    }
          case "1": {
             AddSerialMonitorLine("High level error");
             break;
                    }
            case "2": {
              AddSerialMonitorLine("Low level error");
              break;
                      }
           case "3": {
               AddSerialMonitorLine("Warning"); break; }
                     }
                     return;
          }
       AddSerialMonitorLine(CurrentNumber+ ": "+ res);
       GetNextNumber = true;
      }
}
```

Данный метод принимает в качестве аргумента тип события *SerialDataReceivedEventArgs*. Большинство последовательных соединений используют возврат каретки (\r) или перевод строки (\n), чтобы определить, что команда завершена.

Сначала в строковую переменную res записывается результат выполнения операции манипулятором. И если результат не пустой, и если данные считаны корректно, то выходим из метода и ничего не делаем. Если же ошибка найдена, то обрабатываем исключение с помощью case-метода. Записываем в лог уровень ошибки и выводим на экран. Переводим переменную в значение False, сообщая компилятору что ошибка обработана.

Далее рассмотрим одну из управляющих команд, так, нажатие кнопки +100 X будет иметь следующий программный код:

```
private void button13_Click(object sender, EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
            i = i + 100;
            label8.Text = i.ToString();
                if (serialPort1.IsOpen)
serialPort1.Write("DS 100,0,0" + End);
    }
```

Данный обработчик события нажатия на кнопку, сначала перезапишет в label считанные до этого данные с этой кнопки и конвертирует их в тип Int32, обновляя при этом текущие координаты позиции, добавив к сумме по оси X данные с этой кнопки. И далее проверит, если связь еще существует *if (serialPort1.IsOpen)*, то передаст по этому порту команду смещения манипулятора по оси на эту величину и сам возврат каретки, означающий что команда завершена.

Полный исходный код программы указан в *Приложении А*.

Выводы. В результате работы создано рабочее приложение для удаленного управления роботом-манипулятором RV-2AJ в прямоугольной системе координат.

6. АПРОБИРОВАНИЕ РЕЗУЛЬТАТОВ

Данный программный комплекс прошел успешную проверку на рабочем манипуляторе учебного стенда кафедры (рис. 6.1).

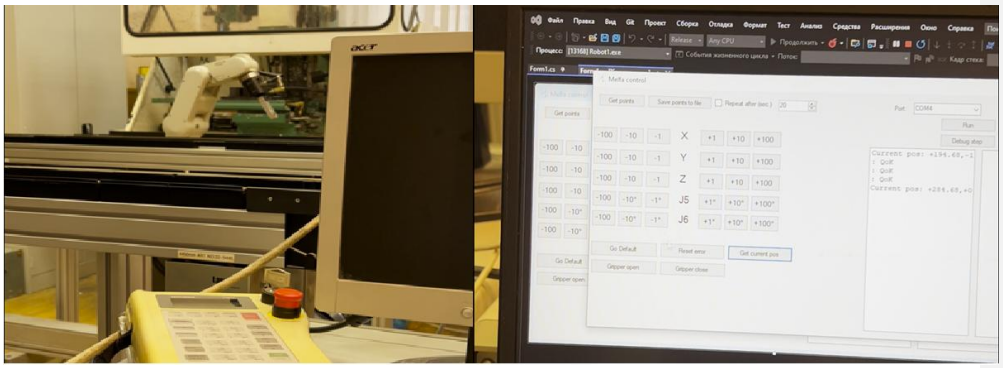

Рисунок 36.1 Апробирование результатов. Процесс передачи команды роботу

Рисунок 6.2 Апробирование результатов. Смена позиции робота

Рисунок 4~~8~~.3 Апробирование результатов. Возврат робота в исходное положение

По итогам апробации подготовлены аналитические материалы, содержащие результаты разработки программы и результатов ее тестирования. Данная ВКР может служить основой для направлений дальнейшей работы по разработке и модификации удаленной управляющей программы робота-манипулятора RV-2AJ в прямоугольной системе координат. В ходе апробации результатов было установлено, что данная программа может быть расширена методом добавления в нее дополнительных программных функций, а также обработке различного рода исключений, возникающих в случае некорректной работы пользователя с ней.

ЗАКЛЮЧЕНИЕ

В результате проделанной работы, были решены следующие задачи:

- ✓ Проведен анализ состояния проблемы по информационным источникам,
- ✓ Проведен анализ устройства робота-манипулятора RV-2AJ и системы координат данного робота.
- ✓ Проведено сравнение функциональных возможностей робота.
- ✓ Проанализирована система команд языка управления роботом.
- ✓ Определены команды, необходимые для управления роботом в прямоугольной системе координат.
- ✓ Выбрана общесистемная методология проектирования.
- ✓ Разработана use-case UML диаграмма вариантов использования.
- ✓ Разработана UML диаграмма компонентов.
- ✓ Представлена IDEF0 нотация.
- ✓ Разработано техническое задание.
- ✓ Разработана функциональная и структурны схемы.
- ✓ Определен метод и протокол подключения ПК к контроллеру робота.
- ✓ Представлен алгоритм работы программы при загрузке системы.
- ✓ Разработан интерфейс пользователя системы.
- ✓ Разработан исходный код.
- ✓ Протестирована работа системы на реальном оборудовании.

СПИСОК ИСПОЛЬЗОВАННЫХ ИСТОЧНИКОВ

1. MITSUBISHI INDUSTRIAL ROBOT MELFA F Series Catalog (English), 2017 [Электронный справочник] – URL: https://www.allied-automation.com/wp-content/uploads/2015/02/MITSUBISHI_catalog_robot_f.pdf (дата обращения: 21.04.2022). – Текст.: электронный.
2. Белоусов И. Р. Алгоритмы управления роботом-манипулятором через Интернет – Общероссийский математический портал, Матем. моделирование, 2002, том 14, номер 8 – с.10 –15 – Текст.: электронный.
3. Описание g-кодов. [сайт] – URL: https://darxton.ru/wiki-article/opisanie-g-kodov/ (дата обращения: 26.04.2022). – Текст.: электронный.
4. Применение нейронной сети для решения обратной задачи кинематики манипулятора Снисаренко С. В. – БГУИР, с. 32, Минск – Международная научно-техническая конференция, 2021 – ISBN 978-985-583-654-5. // МИДО [Электронное издание]. – URL: https://rep.bntu.by/bitstream/handle/data/98113/Informacionno_kommunikacionnye.pdf?sequence=1&isAllowed=y (дата обращения: 25.04.2022) – Текст.: электронный.
5. Mitsubishi Industrial Robot CR1/CR2/CR3/CR4/CR7/CR8/CR9 Controller INSTRUCTION MANUAL Detailed explanations of functions and operations [Электронный справочник] – URL: https://dokumen.tips/documents/mitsubishi-controller-manual-cr1cr2cr3cr4cr7cr8cr9-instruction-manual.html (дата обращения: 25.04.2022) – Текст (перевод).: электронный.
6. Шурков В.Н. Основы автоматизации производства и промышленные роботы М.: Машиностроение, 1989. – 224 с. — ISBN 5-217-00369-3 – Текст: электронный.
7. Гаврилов А.В. Введение в робототехнику Лекция 3. Из чего состоят интеллектуальные роботы– НГТУ [Электронное издание] –

Код поля изменен

URL:http://ermak.cs.nstu.ru/neurotech/html/metodmat/Robots2014/Lect3.pdf(дата обращения: 29.04.2022) – Изображение: электронное.

8. Лычагин Ю.В. Программирование на языке Melfa-Basic IV промышленных роботов фирмы mitsubishi electric практическое пособие / Ю.В. Лычагин; Балт. гос. техн. ун-т. – СПб., 2013. – 41 с
9. Хомченко В.Г., Клевакин В.В., Лазаренко И.В., Горбатых А.С. Обобщенная математическая модель кинематики робота типа RV-2AJ фирмы Mitsubishi Electric – ОГТУ // Омский научный вестник № 1 (107) 2012, с 165.
10. Избачков Ю.С., Петров В.Н., Васильев А. А., Телина И. С. Информационные системы. СПб.: ПИТЕР, 2005. - 65 с. //Учебник для вузов 3-е издание. – Текст: электронный.
11. Лекция 35. Объектно-ориентированное проектирование ЭИС [сайт] – URL: http://kgau.ru/istiki/umk/pis/l35.htm (дата обращения: 30.04.2022) – Текст: электронный.
12. Выбор CASE-средства проектирования информационных систем [сайт] – URL: https://intellect.icu/vybor-case-sredstva-proektirovaniya-informatsionnykh-sistem-5209 (дата обращения: 29.04.2022) – Текст: электронный.
13. Простое руководство по диаграммам компонентов. [сайт] – URL: https://creately.com/blog/ru/uncategorized-ru/учебное-пособие-по-компонентной-диаг/ (дата обращения: 10.05.2022) – Текст: электронный.
14. Кинзябулатов Р. IDEF0. Знакомство с нотацией и пример использования [сайт] – URL: https://trinion.org/blog/idef0-znakomstvo-s-notaciey-i-primer-ispolzovaniya (дата обращения: 11.05.2022) – Текст: электронный.
15. ГОСТ 19.201-78 Техническое задание. Требования к содержанию и оформлению от 18.12.1978
16. ГОСТ 34.601-90. Информационная технология. Комплекс стандартов на автоматизированные системы. Автоматизированные системы. Стадии создания от 01.01.1992

17. Пример технического задания по ГОСТ 19.201-78 [сайт] – URL: https://pro-prof.com/forums/topic/пример-технического-задания-по-гост-19-201-78 (дата обращения: 13.05.2022) – Текст: электронный.

ПРИЛОЖЕНИЕ А

Листинг программы

```
using System;
using System.Collections.Generic;
using System.ComponentModel;
using System.Data;
using System.Drawing;
using System.Linq;
using System.Text;
using System.Windows.Forms;
using System.IO;
using System.Threading;
using System.IO.Ports;
using System.Text.RegularExpressions;

namespace Robot1
{
    public partial class Form1 : Form
    {
        char End = (char)13;
        string CurrentNumber = "";
        bool GetNextNumber = true;
        bool GetWhere = false;
        bool GetError = false;


        public Form1()
        {
            InitializeComponent();
            /*try
            {
                if (!serialPort1.IsOpen)
serialPort1.Open();
            }
            catch { }*/
            //  serialPort1.Write("WH" + End);

            string[] ports = SerialPort.GetPortNames();
            SerialPortsList.Items.Clear();
            SerialPortsList.Items.AddRange(ports);
        }
```

```
        private string GetPosNumber(string Line)
        {
            string result = Line.Substring(2);
            int pos = result.IndexOf('O');
            if (pos > 0) result = result.Substring( pos +
1);
            pos = result.IndexOf(',');
            if (pos > 0) result = result.Substring(0,
pos);
            result = result.Trim();
            return result;
        }
private void button4_Click(object sender, EventArgs e)
        {
            LogList.Items.Clear();
            if (!serialPort1.IsOpen)  serialPort1.Open();
            string[] Lines = Text.Split('\r');
            for (int i = 0; i < Lines.Count(); i++)
            {
                if (Lines[i].IndexOf("MO") > -1)
                {
                    CurrentNumber = GetPosNumber(Lines[i]
+ End);
                    GetNextNumber = false;
                    serialPort1.Write("PR " +
CurrentNumber + End);
                    Thread.Sleep(250);
                    //while (!GetNextNumber) ;
                }

            }
        }

        private void serialPort1_DataReceived(object
sender, System.IO.Ports.SerialDataReceivedEventArgs e)
        {
            string res =
serialPort1.ReadTo(End.ToString());
            if (res != "")
            {
                if(GetWhere)
                {
                    GetWhere = false;
                    AddSerialMonitorLine("Current pos: "
+ res);
```

```
                    return;
                }

                if (GetError)
                {
                    GetError = false;
                    switch (res)
                    {
                        case "0": {
AddSerialMonitorLine("Ok"); break; }
                        case "1": {
AddSerialMonitorLine("High level error"); break; }
                        case "2": {
AddSerialMonitorLine("Low level error"); break; }
                        case "3": {
AddSerialMonitorLine("Warning"); break; }
                    }
                    return;
                }
                AddSerialMonitorLine(CurrentNumber+ ": "+
res);
                GetNextNumber = true;
            }
        }

        delegate void SetTextCallback(string text);
        private void AddSerialMonitorLine(string str)
        {

            if (this.LogList.InvokeRequired)
            {
                SetTextCallback d = new
SetTextCallback(AddSerialMonitorLine);
                this.Invoke(d, new object[] { str });
            }
            else
            {

                this.LogList.Items.Add(str.Trim());
                LogList.SelectedIndex =
LogList.Items.Count - 1;
                LogList.SelectedIndex = -1;
            }

        }
```

```
private void button5_Click(object sender, EventArgs e)
        {
            saveFileDialog2.ShowDialog(this);
            if (saveFileDialog2.FileName != "")
            {

                System.IO.StreamWriter SaveFile = new
System.IO.StreamWriter(saveFileDialog2.FileName);
                foreach (var item in LogList.Items)
                {
                    SaveFile.WriteLine(item);
                }

                SaveFile.Close();

            }

        }

        private void Form1_FormClosing(object sender,
FormClosingEventArgs e)
        {
            serialPort1.Close();
        }

        private void numericUpDown1_ValueChanged(object
sender, EventArgs e)
        {
            timer1.Interval = (int)numericUpDown1.Value *
1000;
        }



        private void checkBox1_CheckStateChanged(object
sender, EventArgs e)
        {
            if (!checkBox1.Checked) timer1.Enabled =
false;
        }

        private void button6_Click(object sender,
EventArgs e)
        {
            label8.Text = "0";
```

```
            label9.Text = "0";
            label10.Text = "0";
            label11.Text = "0";
            label12.Text = "0";
            if (serialPort1.IsOpen) serialPort1.Write("MO
100" + End);
        }

        private void button8_Click(object sender,
EventArgs e)
        {
            if (serialPort1.IsOpen)
serialPort1.Write("GO" + End);
        }

        private void button9_Click(object sender,
EventArgs e)
        {
            if (serialPort1.IsOpen)
serialPort1.Write("GC" + End);
        }

        private void GetErrorFromRobot()
        {
            if (serialPort1.IsOpen)
            {
                GetError = true;
                serialPort1.Write("ER" + End);
            }
        }

        private void button7_Click(object sender,
EventArgs e)
        {
            if (serialPort1.IsOpen)
serialPort1.Write("1;1;RSTALRM+" + End);
        }

        private void
comboBox1_SelectedIndexChanged(object sender, EventArgs
e)
        {
            if (serialPort1.IsOpen) serialPort1.Close();
            serialPort1.WriteTimeout = 1000;
            serialPort1.PortName = SerialPortsList.Text;
```

```
            try
            {
                if (!serialPort1.IsOpen)
serialPort1.Open();
            }
            catch { }
        }

        private void button10_Click(object sender,
EventArgs e)
        {
            if (serialPort1.IsOpen)
            {
                serialPort1.Write("WH" + End);
                GetWhere = true;
            }
        }
private void button11_Click(object sender, EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
            i++;
            label8.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
1,0,0" + End);
        }

        private void button28_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label12.Text);
            i++;
            label12.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
5,1" + End);
        }

        private void button27_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label12.Text);
            i = i + 10;
            label12.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
5,10" + End);
        }
```

```
        private void button26_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label12.Text);
            i = i+ 100;
            label12.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
5,100" + End);
        }

        private void button31_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label12.Text);
            i--;
            label12.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
5,-1" + End);
        }

        private void button30_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label12.Text);
            i=i-10;
            label12.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
5,-10" + End);
        }

        private void button29_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label12.Text);
            i = i - 100;
            label12.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
5,-100" + End);
        }

        private void button37_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label11.Text);
```

```
            i--;
            label11.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
6,-1" + End);
        }

        private void button36_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label11.Text);
            i = i - 10;
            label11.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
6,-10" + End);
        }

        private void button35_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label11.Text);
            i = i - 100;
            label11.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
6,-100" + End);
        }

        private void button34_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label11.Text);
            i++;
            label11.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
6,1" + End);
        }

        private void button33_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label11.Text);
            i=i+10;
            label11.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
6,10" + End);
        }
```

```
        private void button32_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label11.Text);
            i = i + 100;
            label11.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DJ
6,100" + End);
        }

        private void button12_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
            i = i + 10;
            label8.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
10,0,0" + End);
        }

        private void button13_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
            i = i + 100;
            label8.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
100,0,0" + End);
        }

        private void button1_Click_2(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
            i--;
            label8.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
-1,0,0" + End);
        }

        private void button2_Click_1(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
```

```
            i = i - 10;
            label8.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
-10,0,0" + End);
        }

        private void button3_Click_1(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label8.Text);
            i = i - 100;
            label8.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
-100,0,0" + End);
        }

        private void button16_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label9.Text);
            i++;
            label9.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,1,0" + End);
        }

        private void button15_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label9.Text);
            i = i + 10;
            label9.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,10,0" + End);
        }

        private void button14_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label9.Text);
            i = i + 100;
            label9.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,100,0" + End);
        }
```

```
        private void button19_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label9.Text);
            i--;
            label9.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,-1,0" + End);
        }

        private void button18_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label9.Text);
            i = i - 10;
            label9.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,-10,0" + End);
        }

        private void button17_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label9.Text);
            i = i - 100;
            label9.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,-100,0" + End);
        }

        private void button22_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label10.Text);
            i++;
            label10.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,0,1" + End);
        }

        private void button21_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label10.Text);
```

```
            i = i + 10;
            label10.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,0,10" + End);
        }

        private void button20_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label10.Text);
            i = i + 100;
            label10.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,0,100" + End);
        }

        private void button25_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label10.Text);
            i--;
            label10.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,0,-1" + End);
        }

        private void button24_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label10.Text);
            i = i - 10;
            label10.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,0,-10" + End);
        }

        private void button23_Click(object sender,
EventArgs e)
        {
            int i = Convert.ToInt32(label10.Text);
            i = i - 100;
            label10.Text = i.ToString();
            if (serialPort1.IsOpen) serialPort1.Write("DS
0,0,-100" + End);
        }
```

```
private void button39_Click(object sender, EventArgs e)
        {
            button40.BackColor = Color.White;
            button41.BackColor = Color.White;
            button42.BackColor = Color.White;
            button43.BackColor = Color.White;
            button44.BackColor = Color.White;
            (sender as Button).BackColor = Color.Silver;
            if (serialPort1.IsOpen) serialPort1.Write("SP
5" + End);
        }

        private void label7_Click(object sender,
EventArgs e)
        {

        }

        private void button40_Click(object sender,
EventArgs e)
        {
            button39.BackColor = Color.White;
            button41.BackColor = Color.White;
            button42.BackColor = Color.White;
            button43.BackColor = Color.White;
            button44.BackColor = Color.White;
            (sender as Button).BackColor = Color.Silver;
            if (serialPort1.IsOpen) serialPort1.Write("SP
10" + End);
        }
private void button41_Click(object sender, EventArgs e)
        {
            button39.BackColor = Color.White;
            button40.BackColor = Color.White;
            button42.BackColor = Color.White;
            button43.BackColor = Color.White;
            button44.BackColor = Color.White;
            (sender as Button).BackColor = Color.Silver;
            if (serialPort1.IsOpen) serialPort1.Write("SP
15" + End);
        }

        private void button42_Click(object sender,
EventArgs e)
        {
```

```
            button39.BackColor = Color.White;
            button40.BackColor = Color.White;
            button41.BackColor = Color.White;
            button43.BackColor = Color.White;
            button44.BackColor = Color.White;
            (sender as Button).BackColor = Color.Silver;
            if (serialPort1.IsOpen) serialPort1.Write("SP
20" + End);
        }

        private void button43_Click(object sender,
EventArgs e)
        {
            button39.BackColor = Color.White;
            button40.BackColor = Color.White;
            button41.BackColor = Color.White;
            button42.BackColor = Color.White;
            button44.BackColor = Color.White;
            (sender as Button).BackColor = Color.Silver;
            if (serialPort1.IsOpen) serialPort1.Write("SP
25" + End);
        }

        private void button44_Click(object sender,
EventArgs e)
        {
            button39.BackColor = Color.White;
            button40.BackColor = Color.White;
            button41.BackColor = Color.White;
            button42.BackColor = Color.White;
            button43.BackColor = Color.White;
            (sender as Button).BackColor = Color.Silver;
            if (serialPort1.IsOpen) serialPort1.Write("SP
30" + End);
        }
    }
}
```